



THE HONG KONG
POLYTECHNIC UNIVERSITY
香港理工大學



DEPARTMENT OF APPLIED MATHEMATICS

應 用 數 學 系

Quantitative analysis of statistical arbitrage strategy

Cryptocurrency pairs trading strategy

By

HSU Shao Yu

Supervised by

Prof. Yiu Ka-fai

BSc (Hons) in Investment Science

Department of Applied Mathematics

Faculty of Applied Science and Textiles

The Hong Kong Polytechnic University

April 2022

Content

Abstract.....	3
1. Introduction.....	3
2. Literature review	4
2.1 Statistical arbitrage strategy - Pairs trading	4
2.2 Distance approach.....	5
2.3 Cointegration approach	6
2.4 Time series approach, Stochastic control approach, and other approaches	6
3. Methodology	7
3.1 Data source.....	7
3.1.1 Cryptocurrency exchange - Binance.....	7
3.1.2 Choice of the quote currency	7
3.1.3 Data.....	7
3.2 Distance approach.....	8
3.2.1 Basic distance approach	8
3.2.2 Modified distance approach	10
3.3 Cointegration approach	11
3.3.1 Stationary time series and unit-root test.....	11
3.3.2 Engle-Granger two-step method.....	12
3.3.3 Modified cointegration approach	12
3.4 Trading framework	12
4. Results	14
4.1 Result evaluation	14
4.2 Empirical results	14
4.2.1 Distance approach	14
4.2.2 Cointegration approach	15
5. Conclusion	16
References	18
Appendix.....	19
Distance Approach: Basic vs. Modified	19
Cointegration Approach: Basic vs. Modified	22
Python Codes.....	23

Abstract

Purpose - This paper aims to test the profitability of pairs trading in the cryptocurrency market. The pairs are formed by the distance approach and the cointegration approach, and both approaches are compared between the modified method and the original method in terms of returns and risks.

Methodology - The threshold rule depends on the historical mean and standard deviation over the formation period, while the moving average and moving standard deviation are used in the modified method. The Engle-Granger test is applied to identify the univariate cointegration pairs, and the augmented Dickey-Fuller test is employed to recognize the existence of unit root for stationarity.

Findings - The results show the potential profitability of pairs trading in the new and developing cryptocurrency market. Generally, the distance approach turns out to be more profitable than the cointegration approach. Besides, the modification of the threshold calculation for trading signals leverages of the mean-reversion property more efficiently.

Originality - The adjustment on the threshold rules enhances the profitability of pairs trading strategy, providing a broader aspect for future research.

Keywords - cryptocurrency, statistical arbitrage, pairs trading, mean reversion, cointegration

Paper type Research paper

1. Introduction

Cryptocurrency is a class of digital securities based on decentralized blockchain systems, which is originated from a pseudonymous programmer, “Satoshi Nakamoto”, in the first place (*Nakamoto, 2008*). It provides direct online payments between peers without third-party authorization. Yet, it still retains the essential functions of standard currencies, including store of value, unit of account, and medium of exchange. Despite the similar trading principles to traditional securities, the universal and nonstop market enables cryptocurrencies to be accessible to retail investors (*M. Fil and L. Kristoufek, 2020*).

Since the first cryptocurrency, Bitcoin, was issued in 2009, various alternative cryptocurrencies (or “altcoins”) have participated in this potential market. The total capacity of cryptocurrency grew dramatically to nearly \$3 trillion in November 2021 (CoinMarketCap.com). Along with the increasing momentum and refugee capital (or “hot money”) inside the cryptocurrency market, investors began to apply trading strategies that are commonly used in the stock market and foreign exchange market (or “forex”). Some instances of trading strategies are listed in the following. Day trading

implicates making numerous transactions with predetermined stop-loss and take-profit prices within intraday price movements. High-frequency trading (or “HFT”) is a quantitative strategy that trades automatically with developed algorithms. A portfolio is a collection of financial instruments which reduces risks by diversification. Arbitrage implies finding opportunities for profits caused by price inefficiency.

As a novel attraction for investors, the crypto market is highly volatile due to its short history and the lack of cognition. “Fiat currencies have a measurable value, while cryptocurrencies are purely speculative”, stated by Justin Grossbard, the founder of Compare Forex Brokers. The considerable price volatility and the inefficient market make arbitrage a suitable strategy when trading cryptocurrencies. According to Tadi and Kortchemski (2021), there exist two arbitrage opportunities in the crypto market: exchange-to-exchange arbitrage and statistical arbitrage. The exchange-to-exchange arbitrage generates profits by the price inefficiency between different exchanges, while the statistical arbitrage aims attention at the price spread instead.

Pairs trading is one of the statistical arbitrage strategies which was executed for the first time in the 1980s by Nunzio Tatalaglia and his group at Morgan Stanley (Vidyamurthy, 2004). It is a long-term contrarian strategy using mean-reversion on two highly correlated securities. The theory of mean-reversion is that the price spread will eventually revert to the long-run equilibrium; hence there exist potential statistical arbitrage opportunities. Though pairs trading was verified to be highly profitable implemented in US equities (Gatev *et al.*, 2006), the profitability was decreasing in the process of time (Rad, Low, and Faff, 2015). The crypto market is inefficient, illiquid, and volatile, where pairs trading can still be profitable under these properties, yet the performance of pairs trading is still controversial in the crypto market (M. Fil and L. Kristoufek, 2020).

This paper aims to test the profitability of pairs trading on cryptocurrencies by using Distance Approach and Cointegration Approach. Both approaches are demonstrated with basic and modified methods. In Section 2, background information and past literature are reviewed. Thereafter, Section 3 outlines the methodology used, including the four mentioned approaches, the threshold for opening and closing positions, and the backtest of investment return. Then, the data and the descriptive statistics of the strategy outcomes are presented in Section 4. Eventually, Section 5 discusses and concludes the whole project.

2. Literature review

2.1 Statistical arbitrage strategy - Pairs trading

As specified by Aldridge (2013), statistical arbitrage is not an actual arbitrage opportunity since it does not guarantee risk-free profits. However, the pairs trading strategy is based on mean-reversion, which assumes that the price volatility will eventually revert to long-run equilibrium. This property allows pairs trading to produce potential profits with low risk, which is arbitrage-like.

Gatev *et al.* (2006) are one of the ancestors in the domain of pairs trading. They defined it as a two-stage procedure that consists of a pair formation period and a trading period. In the pair formation period, comoving pairs is matched up and sifted out from a basket of financial securities. Thereafter, the price spread within selected pairs are monitored during the trading period. When the value of the spread diverges to some extent, then a long-short position is opened, which means to buy (long) the underperformed components and sell (short) the overperformed ones simultaneously. If the spread reverts to its historical mean or the predetermined trading period is over, then close both the positions and account for the profits.

Krauss, C. (2017) pointed out that pairs trading has relatively small academic literature compared to contrarian and momentum strategies. Nevertheless, these literature can be classified into five main categories: Distance approach, Cointegration approach, Time-series approach, Stochastic control approach, and other approaches.

2.2 Distance approach

Gatev *et al.* (2006) leveraged the baseline distance approach in U.S. equity markets from the 1960's to the 2000's, and proved that pairs trading is profitable and the returns are robust. The pair formation period of the distance approach can be divided into three parts: Price normalization, Euclidean distance calculation, and Pair selection. Since different financial securities have various range of price, it is necessary to normalize the price before analyzing the relationship between pairs. Subsequently, the sum of squared difference (or "SSD"), also known as the Euclidean norm, of each combination of financial securities could be calculated exhaustively. Afterward, we may choose the top pairs with the minimum SSDs for further transactions and evaluation in the trading period, where the threshold for entry and exit is twice the standard deviation in the pair formation period.

Krauss, C. (2017) stated that the distance approach is straightforward and apparent, and hence suitable for studies with massive data. Gatev *et al.* (2006) proved the profitability of the distance approach on the U.S. CRSP from the 1960's to the 2000's. However, Krauss, C. (2017) claimed that choosing Euclidean squared distance as a pair formation method may not be optimal. The reason is that the distance approach aims to find pairs with minimum SSD, which do successfully discover comoving stocks, but the spread of the top pairs might not be able to diverge sufficiently and therefore yields

limited profits. Do and Faff (2010) extend the research period of Gatev *et al.* (2006) on the same stocks; their results indicate the declining profitability of the Euclidean distance approach after 2009.

2.3 Cointegration approach

Vidyamurthy (2004) demonstrated the theoretical framework of pairs trading based on the cointegration approach and became the most cited literature in this field. He suggested choosing the candidate of pairs by utilizing statistical similarity methods and test the preselected pairs with the Engle-Granger cointegration test. In addition, the thresholds for trading signals are determined in nonparametric ways. Referring to Krauss, C. (2017), most studies of cointegration-based pairs trading follow the threshold rule determined by Gatev *et al.* (2006). Rad *et al.* (2015) demonstrate a considerable empirical study of cointegration, combining Gatev *et al.* (2006) and Vidyamurthy (2004) into consideration.

Tadi, M. and Kortchemski, I. (2020) conducted a cointegration-based pairs trading strategy in the cryptocurrency market, and their results perform better than the naïve buy-and hold strategy. Moreover, they show that the maximum drawdown (or “MDD”) of cointegration-based pairs trading is rationally low. Leung, T. and Nguyen, H. (2018) construct cointegrated portfolios of various cryptocurrencies by applying the Johansen test and Engle-Granger test and compare the performance with and without the stop-loss restrictions. The portfolios consist of Bitcoin (BTC), Ethereum (ETH), Bitcoin Cash (BCH), and Litecoin (LTC) are tested to be profitable under different scenarios. Broek and Sharif (2018) split a set of cryptocurrencies based on their fundamental features into four major classes and prove the profitability after forming a cointegration relationship by unit-root test.

Krauss, C. (2017) claimed that the cointegration approach results in a more dependable equilibrium relationship within pairs from the perspective of econometrics, which offset the disadvantage of the distance approach that the top pairs are not optimally mean-reverting.

2.4 Time series approach, Stochastic control approach, and other approaches

The formation period of both studies of time-series and stochastic control approaches are ignored considering the preliminary analyses had already identified the comoving pairs; these approaches mainly focus on identifying the optimal trading signals during the trading period by using Gaussian Markov chain, Ornstein-Uhlenbeck process, Kalman Filter, and regime-switching (Krauss, C. 2017).

There are some other domains that have limited literature, including Machine learning (ML), Principal components analysis (PCA), and the Copula approach. In view

of the purpose of this paper is to test the profitability of pairs trading in the cryptocurrency market, time series, stochastic control, and other approaches are not put into practice.

3. Methodology

3.1 Data source

3.1.1 Cryptocurrency exchange - Binance

Binance is one of the largest cryptocurrency exchanges in terms of trading volume throughout the world. Their users are allowed to download historical data via Binance API. The obtained data includes the timestamp, symbol, open, high, low, close, and volume, in which only the close price was reserved in this paper for analysis.

3.1.2 Choice of the quote currency

Notice that every cryptocurrency could play a role as either base currency or quote currency, and the long-short position taken in the pairs trading strategy could be simplified into a one-way position. Take Bitcoin (BTC), Ethereum (ETH), Tether USD (USDT) as an example: When it comes to the value of BTC/ETH, it means the amount of ETH required to exchange one unit of BTC, where BTC represents the base currency, and ETH represents the quote currency. Suppose the pair of BTC and ETH is formed in the first stage of the pairs trading. When their spread exceeds the trigger threshold, say BTC performs better than ETH, we should take a short position on BTC/USDT and a long position on ETH/USDT. However, it is as same as shorting BTC/ETH with only one position taken.

Despite the simplification of the trade action from two positions into one, it may not be optimal for the sake of comparing the returns between pairs since the profit and loss will be calculated in various of currency for different pairs, respectively. As a result, USDT is elected to be the quote currency for all the other cryptocurrencies in this paper, and the long-short position remains unchanged. The chosen quote currency, Tether USD (USDT), is a stable coin which is designed to stabilize the value by tracking the traditional fiat currency. Since USDT is backed by U.S. dollars, it has an approximate value of USD \$ 1.00.

3.1.3 Data

In the interest of maximizing USDT through the pairs trading strategy, the top 30 USDT-quoted cryptocurrencies which were listed before 2021 are chosen as the sample for analysis, see Table I. For those selected cryptocurrencies, the time series of their prices quoted by USDT from 2020-10-01 to 2022-03-31 (one and a half years) is used in the following analysis, in which the initial 12 months is the pair formation period and the last 6

months is the trading period. The time frame selected is one day; in other words, the price data is the closing price of each day.

Table I.

Top 30 cryptocurrencies ranked by market capacity (Snapshot on 2021-09-26)

Rank	Name	Symbol	Market Cap (\$)	Price (\$)
1	Bitcoin	BTC	813,490,665,612.76	43,208.54
2	Ethereum	ETH	360,390,447,457.18	3,062.27
3	Cardano	ADA	70,755,984,056.52	2.21
4	Binance Coin	BNB	57,869,701,517.48	344.18
5	XRP	XRP	44,135,149,070.75	0.9447
6	Solana	SOL	40,359,412,132.89	135.7
7	Polkadot	DOT	28,470,590,053.41	28.83
8	Dogecoin	DOGE	26,941,578,027.04	0.205
9	Uniswap	UNI	14,697,295,899.01	24.03
10	Avalanche	AVAX	14,728,794,674.21	66.86
11	Terra	LUNA	13,860,154,750.46	34.64
12	Chainlink	LINK	11,194,215,992.03	24.6
13	Algorand	ALGO	10,457,738,363.25	1.77
14	Litecoin	LTC	10,055,671,731.59	150.64
15	Bitcoin Cash	BCH	9,497,413,984.22	503.67
16	Cosmos	ATOM	8,795,276,577.55	39.62
17	Polygon	MATIC	7,493,094,870.54	1.12
18	FTX Token	FTT	6,739,083,004.21	55.86
19	Stellar	XLM	6,557,563,197.65	0.2762
20	Tron	TRX	6,285,721,557.64	0.08772
21	Ethereum Classic	ETC	6,149,353,063.20	47.2
22	Ve Chain	VET	5,879,355,855.08	0.09141
23	Tezos	XTZ	5,614,946,378.19	6.52
24	THETA	THETA	5,099,126,427.22	5.1
25	Elrond	EGLD	4,355,564,609.35	223.34
26	Monero	XMR	4,214,059,861.47	234.11
27	EOS	EOS	3,809,336,683.17	3.97
28	Fantom	FTM	2,997,814,140.70	1.18
29	Kusama	KSM	2,743,933,507.47	323.96
30	Neo	NEO	2,732,106,620.88	38.73

Note. Cryptocurrencies that cannot be quoted by USDT or listed after 2020-10-01 are eliminated.

Source. CoinMarketCap

3.2 Distance approach

3.2.1 Basic distance approach

According to the distance approach utilized by Gatev *et al.* (2006) on U.S. equities, the top ten pairs that carry the minimum sum of squared deviations (SSD), namely the Euclidean distance, between two normalized price series are selected.

Firstly, price normalization is used to uniform the scales for the calculation of analysis between every two cryptocurrencies.

$$P_{normalized} = \frac{P - \mu}{\sigma} \quad (1)$$

, where μ is the mean and σ is the standard deviation throughout the formation period, see Figure 1.

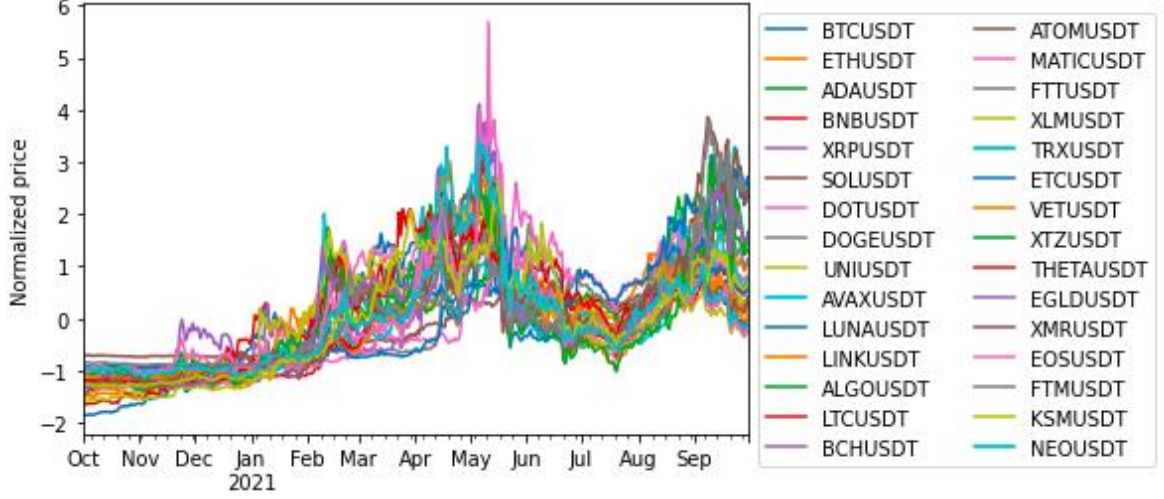


Figure 1. Daily normalized prices

Note. Formation period: October 2020 - September 2021

Then, the Euclidean distances are computed in the following way,

$$SSD_{ij} = \sum_{t=1}^T (P_{i,t} - P_{j,t})^2 \quad (2)$$

, where P_{it} and P_{jt} are the normalized prices of the cryptocurrency i and j at the end of time $t = 1, 2, \dots, T$. Additionally, the empirical mean and standard deviation of the spread $P_i - P_j$ is reserved as the basis and the threshold in the trading period.

$$\mu_{ij} = E[P_i - P_j] \quad (3)$$

$$\begin{aligned} \sigma_{ij} &= \sqrt{Var(P_i - P_j)} = \sqrt{E[(P_i - P_j)^2] - E[P_i - P_j]^2} \\ &= \sqrt{\frac{1}{T} \sum_{t=1}^T (P_{i,t} - P_{j,t})^2 - \left(\frac{1}{T} \sum_{t=1}^T (P_{i,t} - P_{j,t}) \right)^2} \end{aligned} \quad (4)$$

Table II.

Top 10 pairs with minimum Euclidean distance

Rank	Symbol i	Rank	Symbol j	SSD_{ij}	σ_{ij}
26	XMR	30	NEO	26.39319	0.26927
13	ALGO	25	EGLD	26.55623	0.27010

22	VET	30	NEO	28.13674	0.27803
22	VET	26	XMR	28.41008	0.27937
12	LINK	14	LTC	30.79971	0.29089
10	AVAX	16	ATOM	31.37075	0.29357
14	LTC	19	XLM	32.08185	0.29688
4	BNB	22	VET	32.23687	0.29760
8	DOGE	21	ETC	33.16338	0.30184
4	BNB	26	XMR	33.95896	0.30544

3.2.2 Modified distance approach

Considering the trading threshold that is constructed by the empirical mean and standard deviation, the accuracy of mean-reversion will diminish over time. Therefore, a modified distance approach is performed with the moving mean and moving standard deviation instead of the unchanged threshold determined in the formation period. In addition, the price normalization is calculated with moving mean and moving standard deviation likewise.

To obtain the moving mean of the spread, the simple moving average (SMA) is exercised. The SMA used at time t is calculated by summing up the previous data within the time periods,

$$SMA_t(n) = \frac{1}{n} \sum_{h=1}^n (P_{i,t-h} - P_{j,t-h}) \quad (5)$$

, where n is the total number of time periods involved and P_{t-h} is the price h days prior to time t . As well, the moving standard deviation (MSTD) for time t is the standard deviation within the rolling periods,

$$MSTD_t(n) = \sqrt{\frac{\sum_{h=1}^n [(P_{i,t-h} - P_{j,t-h}) - SMA_t]^2}{n}} \quad (6)$$

The period chosen for calculating SMA is determined to be 6 months ($n = 180$), which is nearly the same as the length of the trading period. In other words, the threshold on the last day of the trading period will no longer depend on the price in the pair formation period but on nearly the entire data in the trading period.

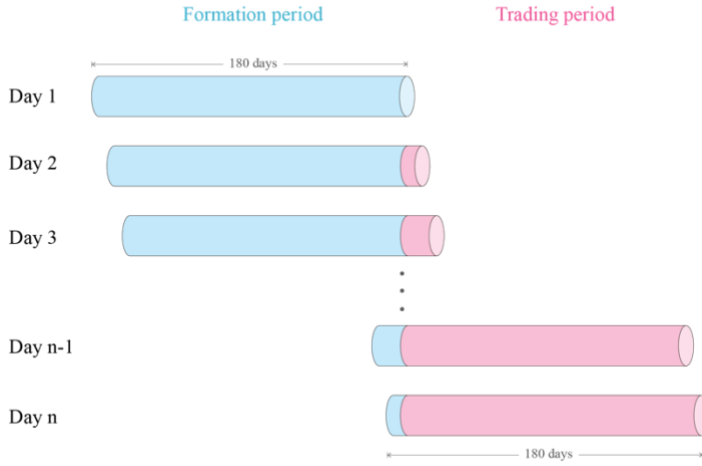


Figure 2. The rolling data for SMA and MSD calculation

Note. The total length is 6 months (180 days) for each period

3.3 Cointegration approach

3.3.1 Stationary time series and unit-root test

A time series $\{X_t\}$ is said to be covariance-stationary (weakly stationary) if and only if:

1. $E[X_t] = \mu$ (constant) for all t ;
2. $Var(X_t) = \sigma^2$ (constant) for all t ;
3. $\gamma_X(t, s)$ only depends on the lag $|t - s|$ and is independent of t and s ,

where $\gamma_X(t, s) = Cov(X_t, X_s)$ is the covariance function of $\{X_t\}$. Since the mean is time-irrelevant, a stationary time series is consequently a mean-reversion process.

The order of integration, denoted $I(d)$, is the minimum times of differentiation needed for a time series to achieve covariance-stationary. If a set of non-stationary time series has the same order of integration $I(d)$ and there exist a linear combination of them that is stationary, then we say the set of time series is cointegrated.

To check whether a time series is stationary or not, unit-root tests could be applied. A unit root is a stochastic trend that make time series to be non-stationary. The existence of the unit-roots may cause crucial problems while analyzing time series, including spurious regressions or errant behavior. Therefore, before adopting cointegration tests, unit-roots tests should be implemented to ensure stationarity. There are several unit-root tests, including the augmented Dickey-Fuller (ADF) test, the Elliot-Granger-Liow test, the Schmidt-Phillips test, the Phillips-Perron (PP) test, and the Zivot-Andrews test. The augmented Dickey-Fuller (ADF) test is used in this paper, in which the null hypothesis and alternative hypothesis are

H_0 : Unit roots exist, the time series is non-stationary

H_1 : No presence of unit roots; the time series is stationary

3.3.2 Engle-Granger two-step method

The Engle-Granger two-step method is to perform a linear regression on the pairs of non-stationary $I(1)$ time series and test if unit-roots exist in the residuals. By the ordinary least squares (OLS) model, cointegration relationships between each pair are built, and the residuals' stationarity are tested by the unit-root test. The model is defined as

$$P_{i,t} = \beta_0 + \beta_1 P_{j,t} + \epsilon_t$$

, where ϵ_t represents the residuals of the cointegrated pair from the OLS. Neglecting the interception β_0 , the spread of the two cryptocurrencies is defined as

$$\epsilon_{ij,t} = P_{i,t} + \beta_1 P_{j,t}$$

, where $P_{i,t}$ and $P_{j,t}$ are the price of cryptocurrency i and j , and γ is the cointegration coefficient. If $P_{i,t}$ and $P_{j,t}$ are $I(1)$ series and $\exists \beta_1$ such that $\epsilon_{ij,t} = P_{i,t} + \beta_1 P_{j,t}$ is an $I(0)$ series, then $P_{i,t}$ and $P_{j,t}$ are cointegrated. (Alexander, 2022)

3.3.3 Modified cointegration approach

Identical with the modification method on the distance approach, simple moving average (SMA) and moving standard deviation is also used to improve the cointegration method.

3.4 Trading framework

The trading strategy for the four mentioned approaches are similar except for the two different threshold determination methods. The initial capital for the backtest is \$1 USDT, and the interim profits are not re-invested. Namely, every single position opened during the trading period is \$1 USDT, which is the same as \$0.5 USDT for both long and short positions. Moreover, there could be only one position opened at a time, and if the spread does not converge at the end of the trading period, the position will be closed automatically.

Table III.

ADF test for order of integration

ADF test	XMR	NEO	ALGO	EGLD	VET	LINK	LTC	AVAX	ATOM	XLM	BNB	DOGE	ETC
t-statistics	-1.97	-1.87	-1.60	-1.17	-1.68	-2.06	-2.31	-0.65	-0.65	-1.97	-1.27	-1.39	-1.37
p-values	0.30	0.34	0.49	0.69	0.44	0.26	0.17	0.86	0.86	0.30	0.64	0.59	0.60
	Δ XMR	Δ NEO	Δ ALGO	Δ EGLD	Δ VET	Δ LINK	Δ LTC	Δ AVAX	Δ ATOM	Δ XLM	Δ BNB	Δ DOGE	Δ ETC
t-statistics	-8.64	-5.75	-6.28	-6.18	-6.79	-23.69	-5.83	-19.19	-7.72	-21.23	-8.95	-5.91	-6.70
p-values	0.01	0.01	0.01	0.01	0.01	0.00	0.01	0.00	0.01	0.00	0.01	0.01	0.01

'1%': -3.45, '5%': -2.87, '10%': -2.57

Table IV.

Engle-Granger two-step method

OLS	XMR ~ NEO	ALGO ~ EGLD	VET ~ NEO	VET ~ XMR	LINK ~ LTC	AVAX ~ ATOM	LTC ~ XLM	BNB ~ VET	DOGE ~ ETC	BNB ~ XMR
β_0	86.1071(***)	0.2501(***)	0.0184(***)	0.0739(***)	2.5960(***)	-6.5157(***)	24.1807(***)	14.7556(***)	-0.0146(***)	230.3592(***)
β_1	3.2766(***)	0.0065(***)	0.0023(***)	0.0007(***)	0.1307(***)	1.8697(***)	428.8504(***)	3079.6331(***)	0.0052(***)	(2.1889(***)
F-statistic	4736(***)	4705(***)	4426(***)	4380(***)	4020(***)	3942(***)	3848(***)	3829(***)	3714(***)	3621(***)
R^2	0.929	0.928	0.924	0.923	0.917	0.916	0.914	0.913	0.911	0.909
ADF for residual										
t-statistics	-2.28	-4.48	-2.91	-3.90	-2.89	-3.86	-4.58	-2.24	-2.88	-3.24
p-value	0.18	0.01	0.04	0.01	0.05	0.01	0.01	0.19	0.05	0.02

(df=1;363) '1%': -3.45, '5%': -2.87, '10%': -2.57

4. Results

4.1 Result evaluation

The backtested results of the basic distance approach, modified distance approach, cointegration approach, and modified cointegration approach are presented in Table V to Table VIII. Additionally, the return charts, together with the spread and signal charts, are attached in Appendix. The result tables consist of rate of return (RoR), standard deviation, Sharpe ratio, maximum drawdown, and return over drawdown (RoMaD).

The rate of return is the cumulative profit or loss throughout the investment period; it is presented in the percentage form among the initial investment.

$$RoR = \frac{\text{current value} - \text{initial value}}{\text{initial value}} \times 100\% \quad (7)$$

Sharpe ratio is the ratio of the average return earned per risk taken, which excess the risk-free rate, in which the risk-free rate is assumed to be zero in this paper.

$$\text{Sharpe ratio} = \frac{RoR - R_f}{\sigma} \quad (8)$$

, where the risk-free rate R_f is assumed to be zero in this paper. Maximum drawdown (MDD) is the largest drop of return from a peak to a trough during the entire investment period, and the return over drawdown (RoMaD) is the average return earned per MDD.

$$MDD = \frac{\text{trough value} - \text{peak value}}{\text{peak value}} \quad (9)$$

$$RoMaD = \frac{RoR}{MDD} \quad (10)$$

4.2 Empirical results

4.2.1 Distance approach

From Table V, half of the pairs selected by the basic distance approach yield positive returns, while the other half generate losses. It is obvious that the number of trades is few, with only one or two trades per pair within the half-year trading period. The overall performance of the basic distance approach is not ideal, with a 50% win rate and insignificant Sharpe ratios.

Table V.
Empirical result of basic distance approach

	No. of trades	Return	STD	Sharpe ratio	MDD	RoMaD
XMRNEO	2	0.37388	0.13479	2.77370	-1.82466	0.20490
ALGOEGLD	1	-0.24635	0.30849	-0.79855	-5.90814	-0.04170
VETNEO	1	-0.02147	0.12104	-0.17742	-1.29803	-0.01654
VETXMR	1	0.40540	0.18336	2.21090	-7.85294	0.05162
LINKLTC	2	0.70808	0.30480	2.32309	-0.88750	0.79784
AVAXATOM	1	-1.04172	0.58663	-1.77576	-22.41185	-0.04648
LTCXLM	1	0.32041	0.16477	1.94457	-5.88056	0.05449
BNBVET	1	-0.13664	0.14363	-0.95134	-3.97234	-0.03440
DOGEETC	1	0.02333	0.00569	4.10260	-1.35780	0.01718
BNBXMR	1	-0.18437	0.19291	-0.95569	-5.42150	-0.03401

Comparing Table VI to Table V, there is a remarkable increase in total trades, and every selected pair makes a positive return ignoring the transaction costs. It is hard to make comments on the MDD difference because there are both raises and drops.

Table VI.
Empirical result of modified distance approach

	No. of trades	Return	STD	Sharpe ratio	MDD	RoMaD
XMRNEO	6	2.18342	0.72874	2.99618	-0.76056	2.87079
ALGOEGLD	1	0.71781	0.35185	2.04007	-0.16532	4.34187
VETNEO	2	0.45704	0.23315	1.96030	-8.62622	0.05298
VETXMR	3	1.30990	0.57969	2.25967	-4.28602	0.30562
LINKLTC	3	1.52099	0.61109	2.48898	-1.13302	1.34242
AVAXATOM	1	0.36750	0.27317	1.34532	-18.21528	0.02018
LTCXLM	1	0.18553	0.22149	0.83763	-0.04402	4.21476
BNBVET	2	0.40100	0.22390	1.79098	-1.52797	0.26244
DOGEETC	3	0.22796	0.21462	1.06215	-0.84361	0.27022
BNBXMR	1	0.00614	0.36577	0.01679	-6.29875	0.00098

As the figures are shown in Appendix, the modified distance approach catches the mean-reversion of the spread superior to the basic distance approach. Hence, there are more trades, and it is more profitable generally.

4.2.2 Cointegration approach

In Table III, the augmented Dickey-Fuller (ADF) test is performed to check the order of integration of the cryptocurrency prices. All the 10 cryptocurrencies are not stationary, while all their first derivatives are stationary with 99% confidence. As a result, it infers that they are $I(1)$ series, which can be tested for cointegration with the Engle-Granger two-step method. The ordinary least squares (OLS) estimation and the unit-root test for residuals are unfolded in Table IV, in which only the four pairs ALGOEGLD, VETXMR, AVAXATOM, LTCXLM are validated to be cointegrated.

In Table VII, the performance of the four cointegrated pairs is not ideal, and the MDDs are relatively large. Moreover, the modification replacement by moving average and moving standard deviation does not raise the return and lower the MDD generally.

Table VII.

Empirical result of basic cointegration approach

	No. of trades	Return	STD	Sharpe ratio	MDD	RoMaD
XMRNEO	-	-	-	-	-	-
ALGOEGLD	2	-0.19852	0.26849	-0.73940	-9.59225	-0.02083
VETNEO	-	-	-	-	-	-
VETXMR	1	0.03963	0.01784	2.22172	-8.74095	0.00453
LINKLTC	-	-	-	-	-	-
AVAXATOM	2	-19.24886	8.25536	-2.33168	nan	nan
LTCXLM	1	0.09091	0.04803	1.89254	-4.22060	0.02154
BNBVET	-	-	-	-	-	-
DOGEETC	-	-	-	-	-	-
BNBXMR	-	-	-	-	-	-

Table VIII.

Empirical result of basic cointegration approach

	No. of trades	Return	STD	Sharpe ratio	MDD	RoMaD
XMRNEO	-	-	-	-	-	-
ALGOEGLD	1	-0.07764	0.24712	-0.31417	-5.78020	-0.01343
VETNEO	-	-	-	-	-	-
VETXMR	1	0.02405	0.00974	2.46934	-2.86547	0.00839
LINKLTC	-	-	-	-	-	-
AVAXATOM	1	-8.86683	6.10380	-1.45267	-21.67499	-0.40908
LTCXLM	1	0.06703	0.03888	1.72408	-4.22060	0.01588
BNBVET	-	-	-	-	-	-
DOGEETC	-	-	-	-	-	-
BNBXMR	-	-	-	-	-	-

5. Conclusion

In this paper, the pairs trading strategy is conducted in the cryptocurrency market in the interest of profitability by means of distance approach and cointegration approach. Top 10 pairs with minimum Euclidean distance in terms of normalized price series are formed, in which the cointegrated pairs are sorted out for additional assessment. The pairs trading strategy takes advantage of the property of mean-reversion for statistical arbitrage opportunity. The entire simulation period is one and a half year, from 2020-10-01 to 2022-03-31, where the pairs are formed with historical data in the first year, and the pairs are traded in the latter half year. The basic trading framework is to open a \$1 USDT position on the spread when it diverges over twice its historical standard

deviation and close it when the spread converges to its historical mean. The chosen of simulation duration and the threshold rule are referred to the work of Gatev *et al.* (2005). On top of the basic threshold method using historical mean and standard deviation in the formation period, the moving average and moving standard deviation are utilized for price normalization and the threshold for trading signals. In accordance with the result, the execution of the modified method could be a double-edged sword. On one hand, the spread becomes more mean-reverting, which is beneficial that the number of trades is increased; On the other hand, the moving mean might cause a negative return in a round of opening and closing the position.

To conclude the performance of the basic and modified methods on distance and cointegration approaches, pairs trading is still profitable in the cryptocurrency market, and the results of the modified distance approach outperformed the others. Nevertheless, the outcomes may alter in different scenarios of threshold rules and backtest period. Future research could aim the attention on comparing different threshold rules or more extended investigation periods, whereas the history length of the cryptocurrency market is still short, and the volatility is not sufficiently stable.

References

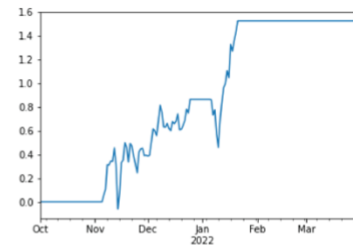
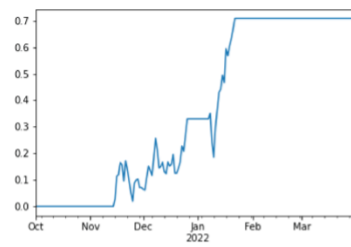
- Aldridge, I. (2013), *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*, John Wiley and Sons.
- Dickey, D.A. and Fuller, W.A. (1979), "Distribution of the estimators for autoregressive time series with a unit root", *Journal of the American Statistical Association*, Vol. 74 No. 366, pp. 427-431.
- Do, B. and Faff, R. (2010) Does simple pairs trading still work? *Financial Analysts Journal* 66(4): 83–95.
- Engle, R.F. and Granger, C.W.J. (1987), "Co-integration and error correction: representation, estimation, and testing", *Econometrica*, Vol. 55 No. 2, pp. 251-276.
- Gatev, E., Goetzmann, W.N. and Rouwenhorst, K.G. (2006) Pairs trading: performance of a relative-value arbitrage rule. *Review of Financial Studies* 19 (3): 797 – 827.
- K. H. Al-Yahyaee, W. Mensi and S.-M. Yoon, "Efficiency multifractality and the long-memory property of the bitcoin market: A comparative analysis with stock currency and gold markets", *Finance Res. Lett.*, vol. 27, pp. 228-234, Dec. 2018.
- Krauss, C. (2017). Statistical Arbitrage Pairs Trading Strategies: Review and Outlook. *Journal of Economic Surveys*, 31(2), 513–545. <https://doi-org.ezproxy.lb.polyu.edu.hk/10.1111/joes.12153>
- Leung, T. and Nguyen, H. (2018) Constructing cointegrated cryptocurrency portfolios for statistical arbitrage
- M. Fil and L. Kristoufek, "Pairs Trading in Cryptocurrency Markets," in *IEEE Access*, vol. 8, pp. 172644-172651, 2020, doi: 10.1109/ACCESS.2020.3024619.
- Nakamoto S. 2008. Bitcoin: a peer-to-peer electronic cash system.
- Rad, H., Low, R.K.Y. and Faff, R.W. (2015) The profitability of pairs trading strategies: distance, cointegration, and copula methods. Working paper, University of Queensland.
- Tadi, M., & Kortchemski, I. (2021). Evaluation of dynamic cointegration-based pairs trading strategy in the cryptocurrency market. *Studies in Economics and Finance* (Charlotte, N.C.), 38(5), 1054-1075.

Appendix

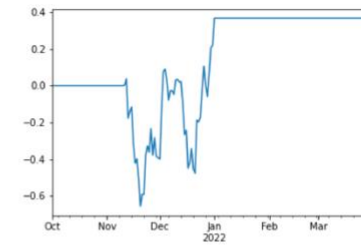
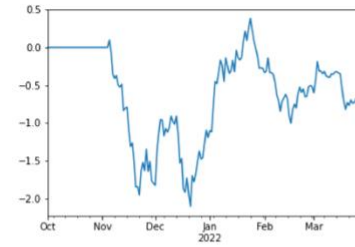
Distance Approach: Basic vs. Modified



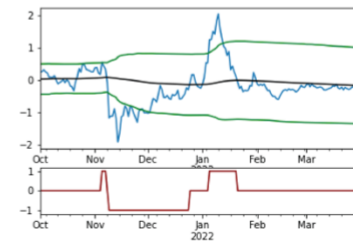
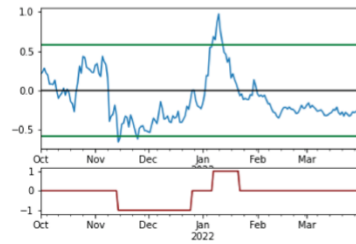
LINKLTC return



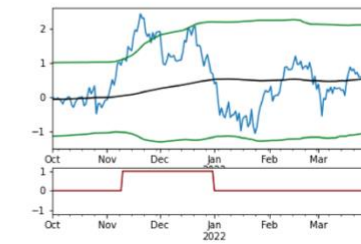
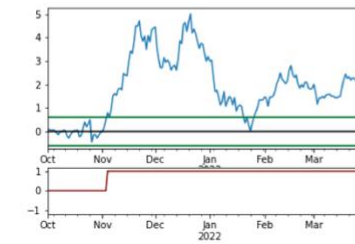
AVAXATOM return



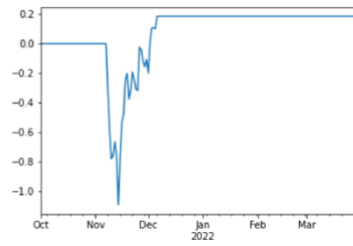
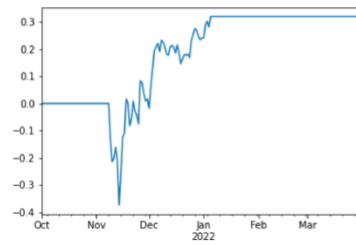
LINKLTC spread & signal



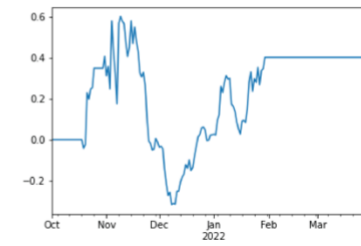
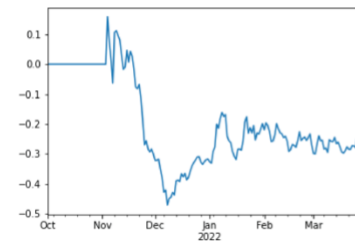
AVAXATOM spread & signal



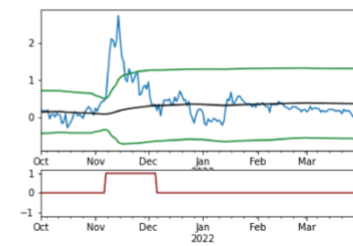
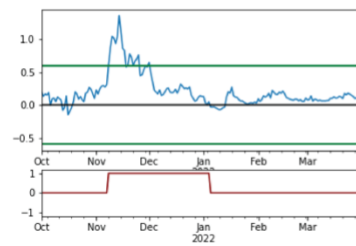
LTCXLM return



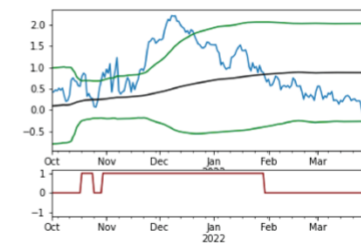
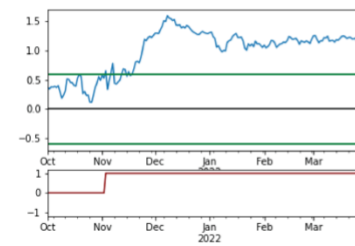
BNBVET return



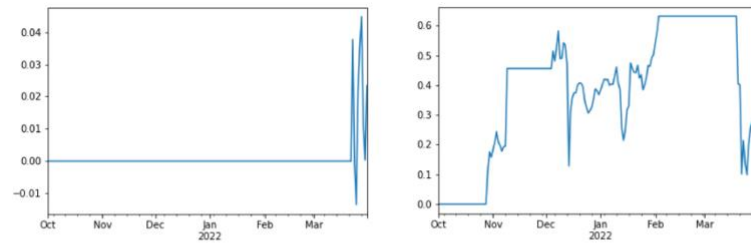
LTCXLM spread & signal



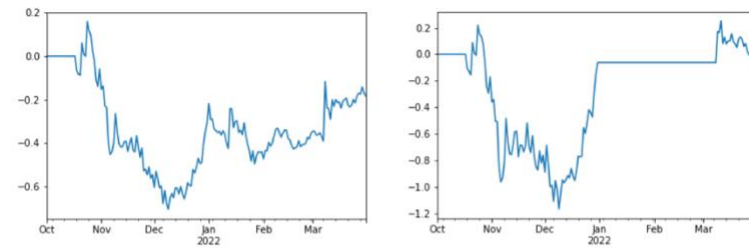
BNBVET spread & signal



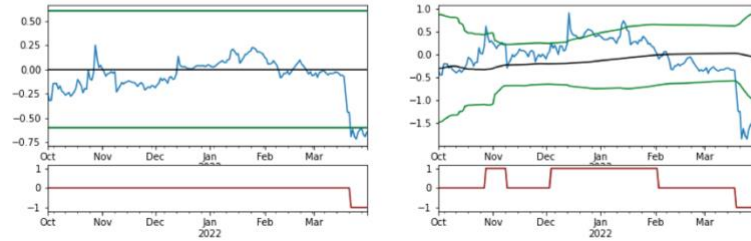
DOGEETC return



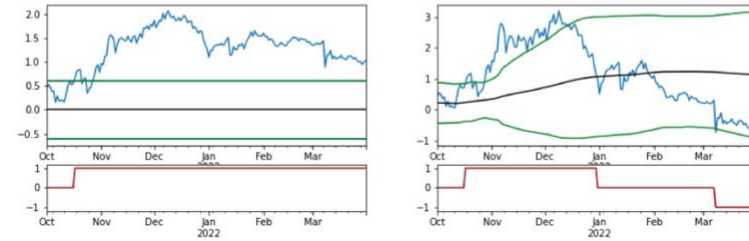
BNBXMR return



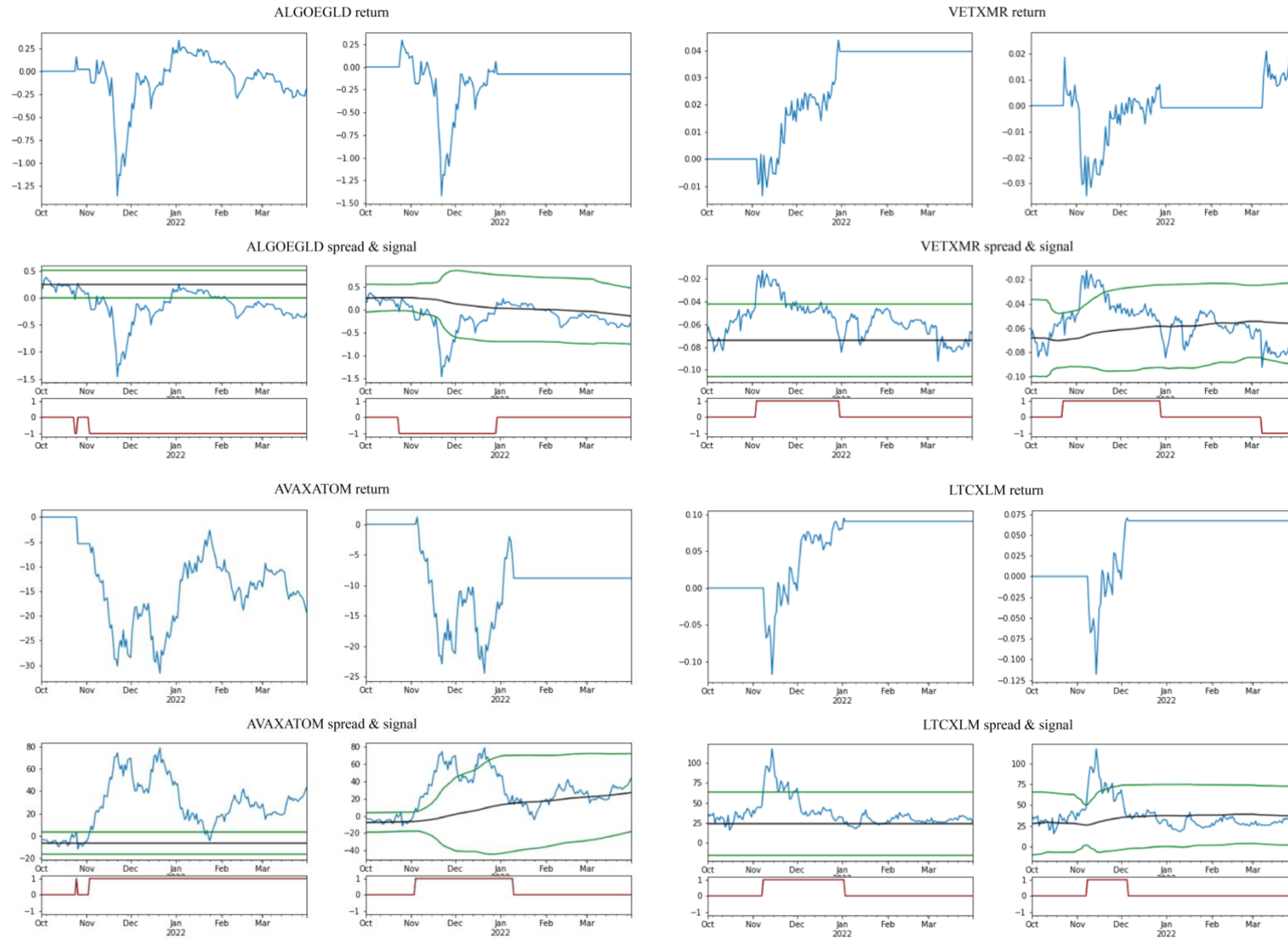
DOGEETC spread & signal



BNBXMR spread & signal



Cointegration Approach: Basic vs. Modified



Python Codes

Import packages

```
import requests
import json
import pandas as pd
import numpy as np
import datetime as dt
import csv
import os

from matplotlib import pyplot as plt
from matplotlib import gridspec
import statsmodels.tsa.stattools as ts
from statsmodels.tsa.vector_ar.vecm import coint_johansen
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

Cryptocurrency symbol/pair list

```
data = open("Symbols.txt", "r")
crypto_list = data.readlines()
crypto_list = [crypto[:-1] for crypto in crypto_list]
pair_list = [crypto + "USDT" for crypto in crypto_list]
print(len(pair_list))
print(pair_list)
data.close()
```

Time intervals

```
interval_list = ['1d', '1h', '1m', '1s']
```

Formation period & trading period

```
start_formation =
str(int(dt.datetime(2020,10,1).timestamp()*1000))
end_formation = str(int(dt.datetime(2021,9,30).timestamp()*1000))
start_trading = str(int(dt.datetime(2021,10,1).timestamp()*1000))
end_trading = str(int(dt.datetime(2022,3,31).timestamp()*1000))
```

Historical data

```
def get_historical_data(symbol, interval, startTime, endTime):
    url = 'https://api.binance.com/api/v3/klines'
```

```

    par = {'symbol': symbol, 'interval': interval, 'startTime':
startTime, 'endTime': endTime}
    df = pd.DataFrame(json.loads(requests.get(url,
params=par).text))
    df.columns = ['datetime', 'open', 'high', 'low', 'price',
'volume', 'close_time', 'qav', 'num_trades', 'taker_base_vol',
'taker_quote_vol', 'ignore']
    df.index = [dt.datetime.fromtimestamp(x//1000.0) for x in
df.datetime]
    df = df.drop(df.columns[[0, 1, 2, 3, 5, 6, 7, 8, 9, 10, 11]],
axis=1)
    df.columns=[symbol]
    df = df.astype(float)
    return df

```

Euclidean distance

```

def get_form_SSD(pair1, pair2):
    spread = norm_form_dfs[pair1] - norm_form_dfs[pair2]
    ssd = np.sum(np.square(spread))
    return ssd

```

Mean & standard deviation

```

def get_form_std(pair1, pair2):
    spread = norm_form_dfs[pair1] - norm_form_dfs[pair2]
    std = spread.std()
    return std

def get_moving_std(pair1, pair2):
    spread = moving_norm_full_dfs[pair1] -
moving_norm_full_dfs[pair2]
    std = spread.rolling(window=180).std().dropna()
    return std

def get_moving_average(pair1, pair2):
    spread = moving_norm_full_dfs[pair1] -
moving_norm_full_dfs[pair2]
    mean = spread.rolling(window=180).mean().dropna()
    return mean

```

Price normalization

```
form_dfs= get_historical_data(pair_list[0], interval_list[0],
start_formation, end_formation)
for i in range(len(pair_list)-1):

form_dfs=pd.DataFrame.join(form_dfs,get_historical_data(pair_list
[i+1], interval_list[0], start_formation, end_formation))
norm_form_dfs=(form_dfs-form_dfs.mean())/form_dfs.std()
norm_form_dfs.plot(ylabel='Normalized price')
plt.legend(bbox_to_anchor=(1.0, 1.0),ncol=2)

trade_dfs = get_historical_data(pair_list[0], interval_list[0],
start_trading, end_trading)
for i in range(len(pair_list)-1):

trade_dfs=pd.DataFrame.join(trade_dfs,get_historical_data(pair_li
st[i+1], interval_list[0], start_trading, end_trading))

norm_trade_dfs=(trade_dfs-form_dfs.mean())/form_dfs.std()
norm_trade_dfs.plot(ylabel='Normalized price')
plt.legend(bbox_to_anchor=(1.0, 1.0),ncol=2)

full_dfs = pd.concat([form_dfs, trade_dfs])

moving_norm_trade_dfs=(trade_dfs-
full_dfs.rolling(window=180).mean())/full_dfs.rolling(window=180)
.std()
moving_norm_trade_dfs = moving_norm_trade_dfs.dropna()
moving_norm_trade_dfs.plot(ylabel='Normalized price')
plt.legend(bbox_to_anchor=(1.0, 1.0),ncol=2)

moving_norm_full_dfs=(full_dfs-
full_dfs.rolling(window=180).mean())/full_dfs.rolling(window=180)
.std()
moving_norm_full_dfs = moving_norm_full_dfs.dropna()
moving_norm_full_dfs.plot(ylabel='Normalized price')
plt.legend(bbox_to_anchor=(1.0, 1.0),ncol=2)
```

Distance approach

Top 10 pairs with minimum SSD

```
top_ssd = []
top_ssd_pair=[]
for i in range (30):
    for j in range(i+1, 30):

top_ssd.append([i,crypto_list[i],j,crypto_list[j],get_form_SSD(pair_list[i], pair_list[j]),get_form_std(pair_list[i], pair_list[j]))

top_ssd_pair.append([pair_list[i],pair_list[j],get_form_SSD(pair_list[i], pair_list[j]),get_form_std(pair_list[i], pair_list[j]))
    top_ssd.sort(key = lambda x: x[-2])
    top_ssd_pair.sort(key=lambda x: x[-2])
print(top_ssd[:10])
print(top_ssd_pair[:10])
top_ssd_list=top_ssd[:10]
```

Return, STD

```
def get_results(pair):
    returns = pair[-1]
    std = pair.std()
    roll_max = pair.rolling(window=len(pair),min_periods=1).max()
    daily_drawdown = (pair-roll_max)/roll_max
    max_daily_drawdown =
daily_drawdown.rolling(window=len(pair),min_periods=1).min()
    mdd = max_daily_drawdown.min()
    sharpe = returns / std
    romad = -(returns/mdd)
    print(returns, std, sharpe, mdd, romad)
```

Basic distance approach

Trading period

```
def get_norm_trade_data(top_ssd_pair):
    r =
norm_trade_dfs[top_ssd_pair[0]].to_frame().join(norm_trade_dfs[top_ssd_pair[1]].to_frame())
```

```

    return r

XMRNEO = get_norm_trade_data(top_ssd_pair[0])
ALGOEGLD = get_norm_trade_data(top_ssd_pair[1])
VETNEO = get_norm_trade_data(top_ssd_pair[2])
VETXMR = get_norm_trade_data(top_ssd_pair[3])
LINKLTC = get_norm_trade_data(top_ssd_pair[4])
AVAXATOM = get_norm_trade_data(top_ssd_pair[5])
LTCXLM = get_norm_trade_data(top_ssd_pair[6])
BNBVET = get_norm_trade_data(top_ssd_pair[7])
DOGEETC = get_norm_trade_data(top_ssd_pair[8])
BNBXMR = get_norm_trade_data(top_ssd_pair[9])

```

Spread

```

XMRNEO_spread= XMRNEO['XMRUSD']-XMRNEO['NEOUSD']
ALGOEGLD_spread = ALGOEGLD['ALGOUSD']-ALGOEGLD['EGLDUSD']
VETNEO_spread = VETNEO['VETUSD']-VETNEO['NEOUSD']
VETXMR_spread = VETXMR['VETUSD']-VETXMR['XMRUSD']
LINKLTC_spread = LINKLTC['LINKUSD']-LINKLTC['LTCUSD']
AVAXATOM_spread = AVAXATOM['AVAXUSD']-AVAXATOM['ATOMUSD']
LTCXLM_spread = LTCXLM['LTCUSD']-LTCXLM['XLMUSD']
BNBVET_spread = BNBVET['BNBUSD']-BNBVET['VETUSD']
DOGEETC_spread = DOGEETC['DOGEUSD']-DOGEETC['ETCUSD']
BNBXMR_spread = BNBXMR['BNBUSD']-BNBXMR['XMRUSD']

```

Trading signals & returns

```

spread = XMRNEO['XMRUSD']-XMRNEO['NEOUSD']
signal = XMRNEO['XMRUSD']-XMRNEO['NEOUSD']
returns = XMRNEO['XMRUSD']-XMRNEO['NEOUSD']
pos = 0
n = 0

for i in range(len(spread)):
    signal[0] = 0
    returns[0] = 0
    if pos == 0 and abs(spread[i]) > 2 * (top_ssd[0][5]):
        pos = np.sign(spread[i])
        returns[i] = returns[i-1]

```

```

    signal[i] = np.sign(signal[i])
    n += 1
    elif pos != 0:
        returns[i] = returns[i-1] + np.sign(spread[i-1]) * (spread[i-1] - spread[i])/2
        signal[i] = np.sign(signal[i])
        if np.sign(spread[i] * np.sign(spread[i-1])) < 0:
            pos = 0
            signal[i] = 0
        else:
            returns[i] = returns[i-1]
            signal[i] = signal[i-1]
returns.plot()
plt.savefig('XMRNEO_return')
print(n)

```

```

get_results(returns)

```

```

fig = plt.figure()
gs=gridspec.GridSpec(2, 1, height_ratios=[3, 1])
ax0=plt.subplot(gs[0])
spread.plot()
plt.axhline(y=2*top_ssd[9][5], color='g',linewidth=2)
plt.axhline(y=0, color='black')
plt.axhline(y=-2*top_ssd[9][5], color='g',linewidth=2)

```

```

ax1=plt.subplot(gs[1])
signal.plot(ylim=[-1.2,1.2],color='darkred')

```

```

plt.savefig('BNBXMR_spread')

```

Modified distance approach

Trading period data

```

def get_moving_norm_trade_data(top_ssd_pair):
    r =
    moving_norm_trade_dfs[top_ssd_pair[0]].to_frame().join(moving_norm_trade_dfs[top_ssd_pair[1]].to_frame())
    return r

```

```

XMRNEO_m = get_moving_norm_trade_data(top_ssd_pair[0])
ALGOEGLD_m = get_moving_norm_trade_data(top_ssd_pair[1])
VETNEO_m = get_moving_norm_trade_data(top_ssd_pair[2])
VETXMR_m = get_moving_norm_trade_data(top_ssd_pair[3])
LINKLTC_m = get_moving_norm_trade_data(top_ssd_pair[4])
AVAXATOM_m = get_moving_norm_trade_data(top_ssd_pair[5])
LTCXLM_m = get_moving_norm_trade_data(top_ssd_pair[6])
BNBVET_m = get_moving_norm_trade_data(top_ssd_pair[7])
DOGEETC_m = get_moving_norm_trade_data(top_ssd_pair[8])
BNBXMR_m = get_moving_norm_trade_data(top_ssd_pair[9])

```

Spread

```

XMRNEO_spread_m = XMRNEO_m['XMRUSDT']-XMRNEO_m['NEOUSDT']
ALGOEGLD_spread_m = ALGOEGLD_m['ALGOUSDT']-ALGOEGLD_m['EGLDUSDT']
VETNEO_spread_m = VETNEO_m['VETUSDT']-VETNEO_m['NEOUSDT']
VETXMR_spread_m = VETXMR_m['VETUSDT']-VETXMR_m['XMRUSDT']
LINKLTC_spread_m = LINKLTC_m['LINKUSDT']-LINKLTC_m['LTCUSDT']
AVAXATOM_spread_m = AVAXATOM_m['AVAXUSDT']-AVAXATOM_m['ATOMUSDT']
LTCXLM_spread_m = LTCXLM_m['LTCUSDT']-LTCXLM_m['XLMUSDT']
BNBVET_spread_m = BNBVET_m['BNBUSDT']-BNBVET_m['VETUSDT']
DOGEETC_spread_m = DOGEETC_m['DOGEUSDT']-DOGEETC_m['ETCUSDT']
BNBXMR_spread_m = BNBXMR_m['BNBUSDT']-BNBXMR_m['XMRUSDT']

```

Trading signals & returns

```

mean = get_moving_average('BNBUSDT', 'XMRUSDT')
std = get_moving_std('BNBUSDT', 'XMRUSDT')
spread = BNBXMR_m['BNBUSDT']-BNBXMR_m['XMRUSDT']
signal = BNBXMR_m['BNBUSDT']-BNBXMR_m['XMRUSDT']
returns = BNBXMR_m['BNBUSDT']-BNBXMR_m['XMRUSDT']
pos = 0
n = 0
for i in range(len(spread)):
    signal[0] = 0
    returns[0] = 0
    if pos == 0 and abs(spread[i]-mean[i+7]) > 2 * std[i+7]:
        pos = np.sign(spread[i]-mean[i+7])
        returns[i] = returns[i-1]
        signal[i] = np.sign(spread[i]-mean[i-7])

```

```

    n += 1
    elif pos != 0:
        returns[i] = returns[i-1] + np.sign(spread[i-1]-mean[i+7]) *
(spread[i-1] - spread[i])/2
        signal[i] = np.sign(spread[i]-mean[i-7])
        if np.sign(spread[i]-mean[i+7]) * np.sign(spread[i-1]-
mean[i+7]) < 0:
            pos = 0
            signal[i] = 0
    else:
        returns[i] = returns[i-1]
        signal[i] = signal[i-1]
returns.plot()
plt.savefig('BNBXMR_return_m')
print(n)

```

```

get_results(returns)

```

```

fig = plt.figure()
gs=gridspec.GridSpec(2, 1, height_ratios=[3, 1])
ax0=plt.subplot(gs[0])
spread.plot()
(mean+2*std)[7:].plot(color='g')
(mean-2*std)[7:].plot(color='g')
mean[7:].plot(color='black')

```

```

ax1=plt.subplot(gs[1])
signal.plot(ylim=[-1.2,1.2],color='darkred')

```

```

plt.savefig('BNBXMR_spread_m')

```

Cointegration

Unit-root test

```

print(ts.adfuller(form_dfs['ETCUSDT'])[2])
print(ts.adfuller(form_dfs['ETCUSDT'].diff()[1:])[2])

```

OLS

```

OLS = smf.ols(formula='DOGEUSDT ~ ETCUSDT',data=form_dfs)

```

```

print(OLS.fit().summary())
b0=OLS.fit().params[0]
b1=OLS.fit().params[1]
ts.adfuller(form_dfs['DOGEUSD'].subtract(b1*form_dfs['ETCUSD']+
b0))[2]

```

Trading signals & returns

```

mean =
form_dfs['ALGOUSD'].subtract(b1*form_dfs['EGLDUSD']).mean()
std =
form_dfs['ALGOUSD'].subtract(b1*form_dfs['EGLDUSD']).std()
spread = trade_dfs['ALGOUSD'].subtract(b1*trade_dfs['EGLDUSD'])
signal = trade_dfs['ALGOUSD'].subtract(b1*trade_dfs['EGLDUSD'])
returns =
trade_dfs['ALGOUSD'].subtract(b1*trade_dfs['EGLDUSD'])
pos = 0
n = 0
for i in range(len(spread)):
    signal[0] = 0
    returns[0] = 0
    if pos ==0 and abs(spread[i]-mean) > 2*std :
        pos = np.sign(spread[i]-mean)
        returns[i] = returns[i-1]
        signal[i] = np.sign(spread[i]-mean)
        n += 1
    elif pos != 0:
        returns[i] = returns[i-1] + np.sign(spread[i]-mean) *
(spread[i-1]-spread[i])/(1+b1)
        signal[i] = np.sign(spread[i]-mean)
        if np.sign(spread[i]-mean) * np.sign(spread[i-1]-mean) <0:
            pos = 0
            signal[i] = 0
    else:
        returns[i] = returns[i-1]
        signal[i] = signal[i-1]
returns.plot()
plt.savefig('ALGOEGLD_return_c')
print(n)

```

```

get_results(returns)

fig = plt.figure()
gs = gridspec.GridSpec(2, 1, height_ratios=[3,1])
ax0 = plt.subplot(gs[0])
spread.plot()
plt.axhline(y=mean, color='k')
plt.axhline(y=mean + 2*std, color='g')
plt.axhline(y=mean - 2*std, color='g')

ax1=plt.subplot(gs[1])
signal.plot(ylim=[-1.2,1.2],color='darkred')

plt.savefig('ALGOEGLD_spread_c')

```

Modified cointegration approach

```

mean =
full_dfs['LTCUSDT'].subtract(b1*full_dfs['XLMUSDT']).rolling(wind
ow=180).mean().dropna()[186:]
std =
full_dfs['LTCUSDT'].subtract(b1*full_dfs['XLMUSDT']).rolling(wind
ow=180).std().dropna()[186:]
spread = trade_dfs['LTCUSDT'].subtract(b1*trade_dfs['XLMUSDT'])
signal = trade_dfs['LTCUSDT'].subtract(b1*trade_dfs['XLMUSDT'])
returns = trade_dfs['LTCUSDT'].subtract(b1*trade_dfs['XLMUSDT'])
pos = 0
n = 0
for i in range(len(spread)):
    signal[0] = 0
    returns[0] = 0
    if pos ==0 and abs(spread[i]-mean[i]) > 2*std[i] :
        pos = np.sign(spread[i]-mean[i])
        returns[i] = returns[i-1]
        signal[i] = np.sign(spread[i]-mean[i])
        n += 1
    elif pos != 0:
        returns[i] = returns[i-1] + np.sign(spread[i]-mean[i]) *
(spread[i-1]-spread[i])/(1+b1)

```

```

    signal[i] = np.sign(spread[i]-mean[i])
    if np.sign(spread[i]-mean[i]) * np.sign(spread[i-1]-mean[i])
<0:
        pos = 0
        signal[i] = 0
    else:
        returns[i] = returns[i-1]
        signal[i] = signal[i-1]
returns.plot()
# plt.savefig('LTCXLM_return_mc')
print(n)

get_results(returns)

fig = plt.figure()
gs = gridspec.GridSpec(2, 1, height_ratios=[3,1])
ax0 = plt.subplot(gs[0])
spread.plot()
(mean+2*std).plot(color='g')
(mean-2*std).plot(color='g')
mean.plot(color='k')
ax1=plt.subplot(gs[1])
signal.plot(ylim=[-1.2,1.2],color='darkred')

plt.savefig('LTCXLM_spread_mc')

```